

Litigo — Moss WASM Technical Deep-Dive Q&A

PART 2: TECHNICAL Q&A

YC Fall 2026 x Moss Zero-Latency Hackathon | Live App: <https://litigo-ai.vercel.app>

TECH Q1 "How is Moss WASM compiled and instantiated in Chrome Extension MV3?"

Litigo compiles vector array math routines into a WebAssembly binary `moss.wasm` and loads it inside the Chrome MV3 background service worker via `chrome.runtime.getURL('moss.wasm')`.

Manifest V3 compliance is guaranteed using local WASM policy: `"extension_pages": "script-src 'self' 'wasm-unsafe-eval'"`.

TECH Q2 "How does Litigo achieve sub-3ms vector similarity queries?"

Rule texts and document chunks are converted into 128-dimensional sparse vector representations in WASM linear memory (`wasmMemory`). When evaluation occurs, `moss.query('litigo_rules', text)` runs parallel array dot-products in WebAssembly, computing cosine scores in **0.89ms**.

TECH Q3 "Explain Litigo's 4-Layer Execution Pipeline step-by-step."

- **Layer 1 (Pre-Injection)**: Intercepts form submission and appends `[ENFORCE: rule1; rule2]` silently to the prompt payload.
- **Layer 2 (Streaming Truncator)**: Listens to DOM mutations via `MutationObserver`. If token count breaches length bounds mid-stream, visible text is truncated immediately with a warning badge.
- **Layer 3 (Moss WASM Semantic Engine)**: Runs parallel WASM cosine queries (confidence > 0.70) against keyword pattern matchers.
- **Layer 4 (Truth Layer Grounding)**: Matches claim vectors against local IndexedDB document vectors:
 - **Verified (●)**: Cosine score ≥ 0.85
 - **Unverified (●)**: Cosine score 0.60 – 0.85
 - **Contradiction (●)**: Cosine score < 0.60

TECH Q4 "Does running WASM in content scripts block the UI thread or cause lag?"

No. WASM vector calculations complete in under 1ms off the main loop inside asynchronous microtasks. DOM badge updates are throttled via `requestAnimationFrame`, keeping the page rendering at a smooth 60 FPS.

**TECH
Q5****"How does the Next.js Enterprise Dashboard communicate with the Chrome Extension?"**

The Next.js 14 App Router dashboard encrypts rule payloads at rest using `AES-256 WebCrypto` (`window.crypto.subtle`). Encrypted rule payloads sync securely into `chrome.storage.local` and `IndexedDB` with zero cloud telemetry or data leakage.

TECH Q6**"What happens if a user inputs conflicting rules?"**

Litigo's dual validation path scores rules independently. If Rule A passes and Rule B fails, Litigo flags the specific rule failure on the inline badge and computes a weighted compliance score: `Math.max(0, 100 - (violations * 15))`.